CS11313 - Spring 2025

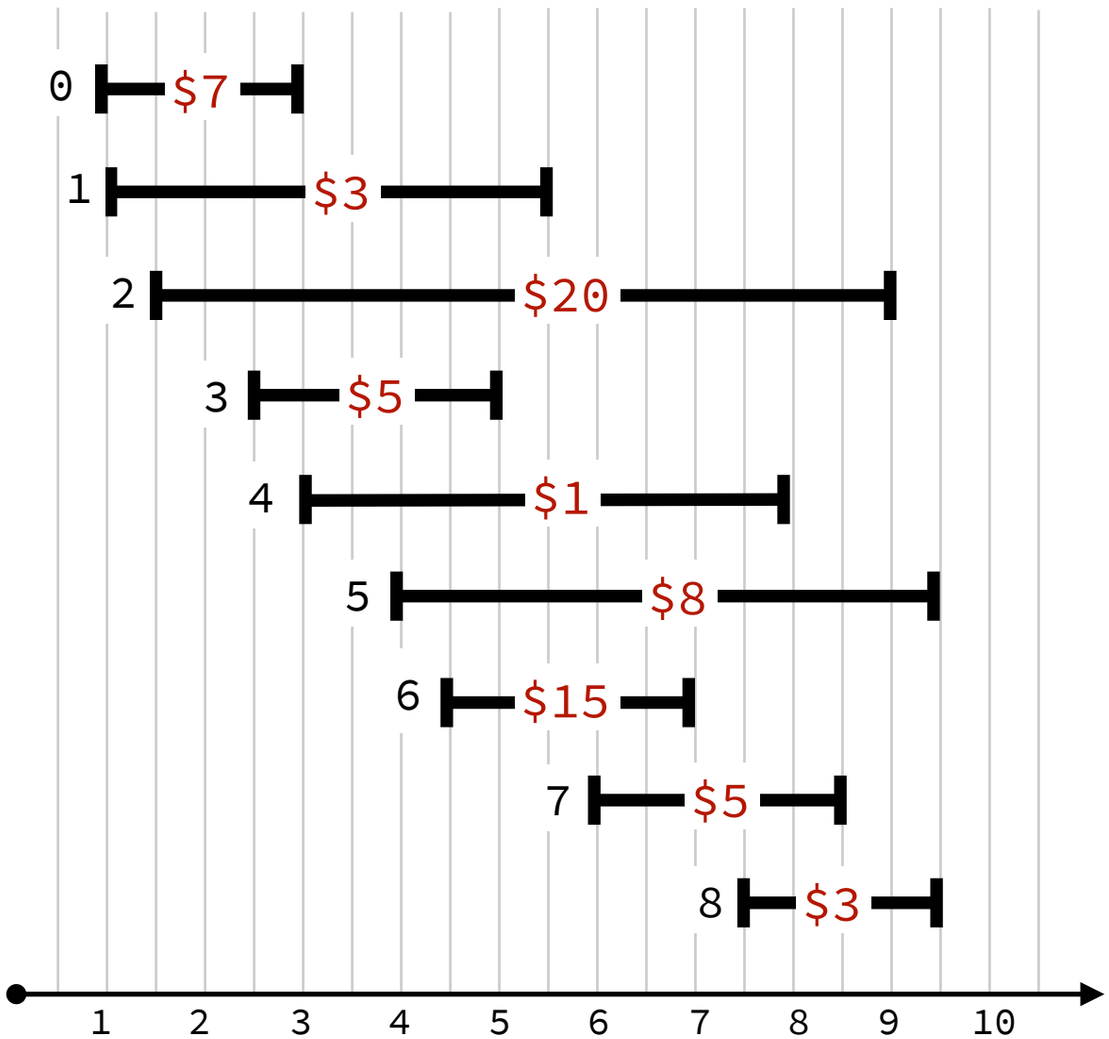
Design & Analysis of Algorithms

Weighted Activity Selection

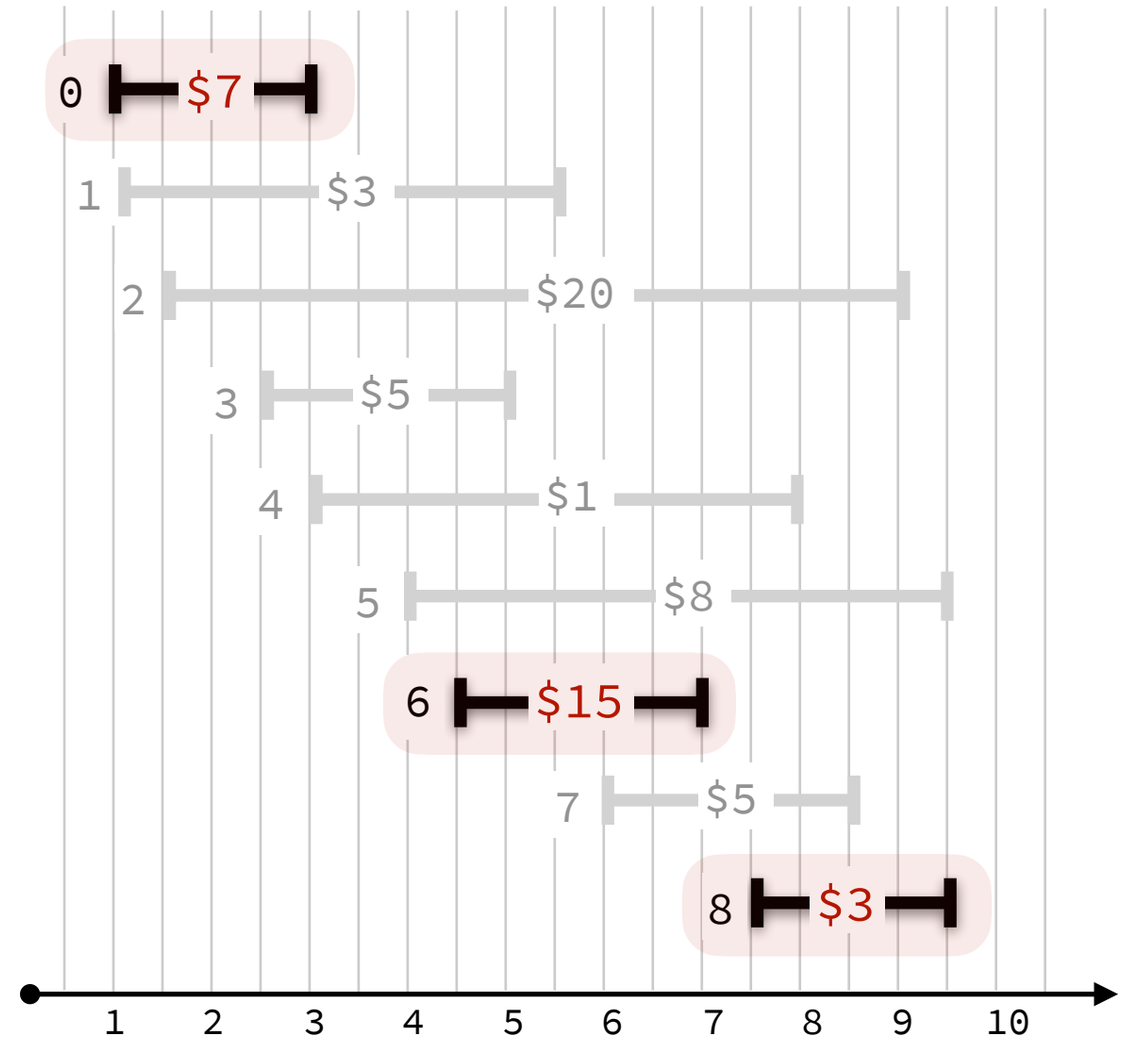
Ibrahim Albluwi

Overview

Problem Statement. Consider a set of n activities, each with a start time, a finish time and a positive value. Find a subset of non-overlapping activities that maximizes the sum of values.

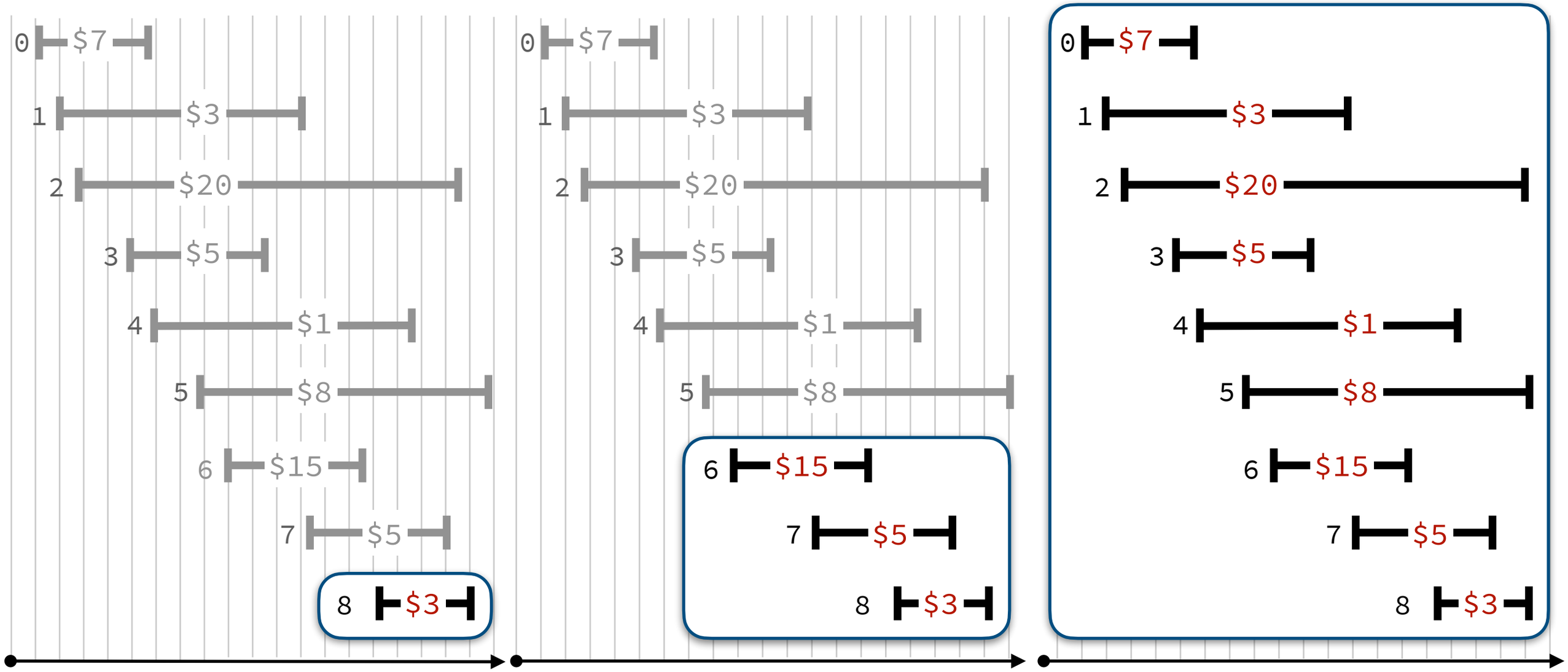


Example. Part-time jobs, where the goal is to pick a subset with the maximum pay (a job cannot be taken partially, and jobs cannot overlap.)



Non-overlapping
Subset with *max* value

1. Defining Subproblems



OPTIMAL(8)
Consider activities 8 → 8

OPTIMAL(6)
Consider activities 6 → 8

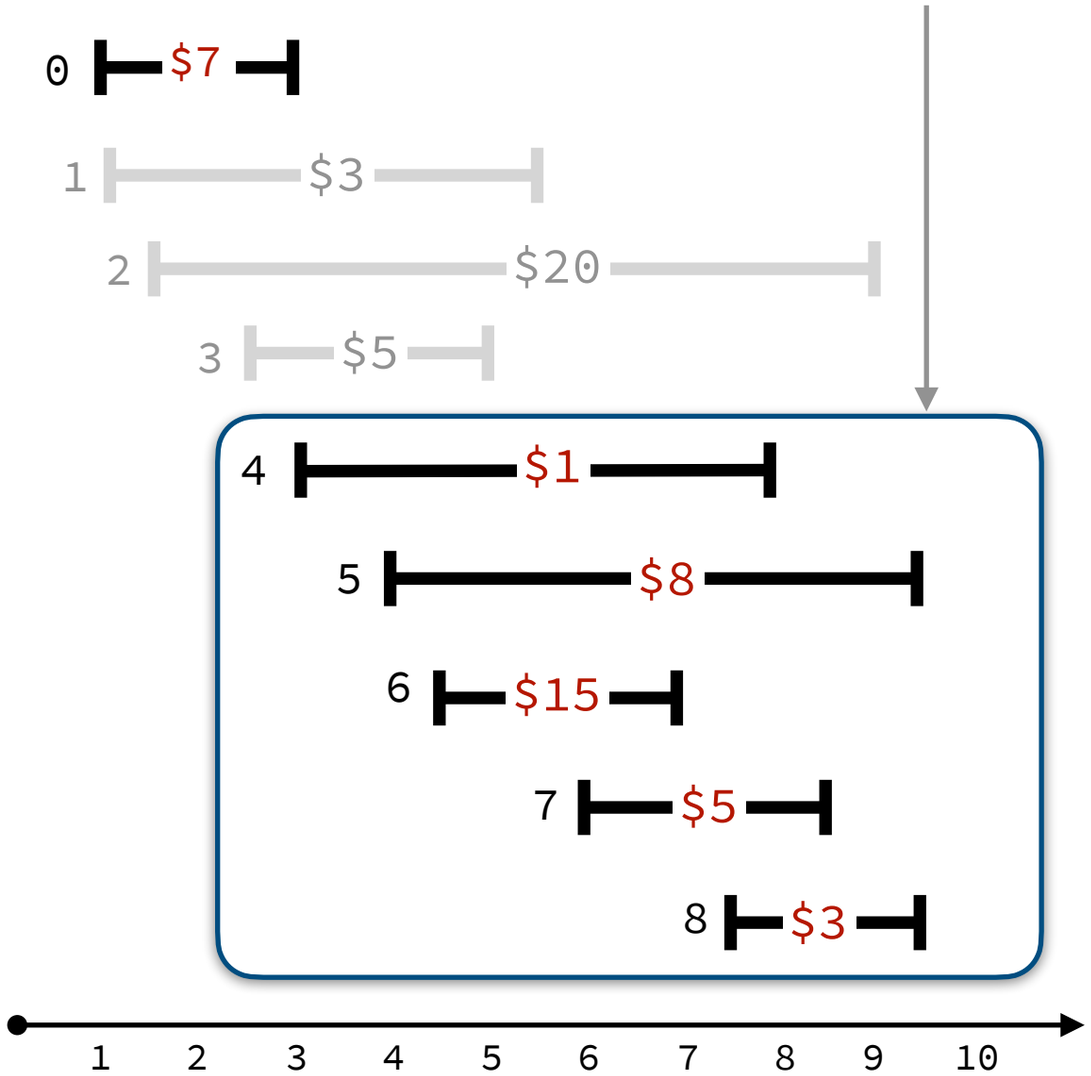
OPTIMAL(0)
Consider activities 0 → 8

OPTIMAL(i) : Subproblem considering activities i → N−1

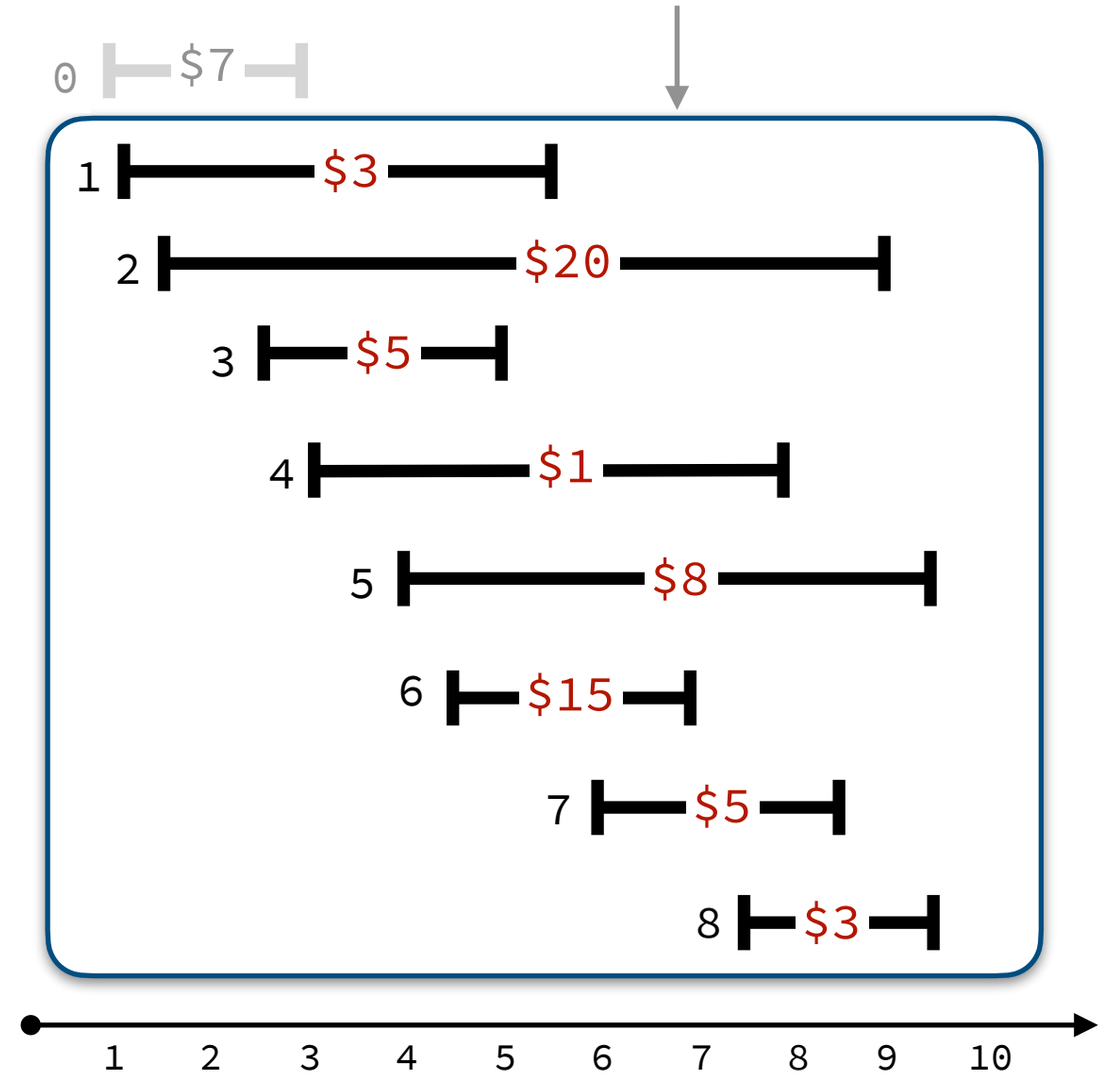
2. Constructing a Recursive Definition

Two possible decisions

Take activity 0 = $7 + Solve this subproblem



Don't take activity 0 = Solve this subproblem




```
OPTIMAL(i) = 0                                          if i >= n
             MAX(OPTIMAL(NEXT(i)) + v[i],
                 OPTIMAL(i+1)     + 0)                   if i <  n
```

3. Check for Overlapping Subproblems

Consider the following instance:

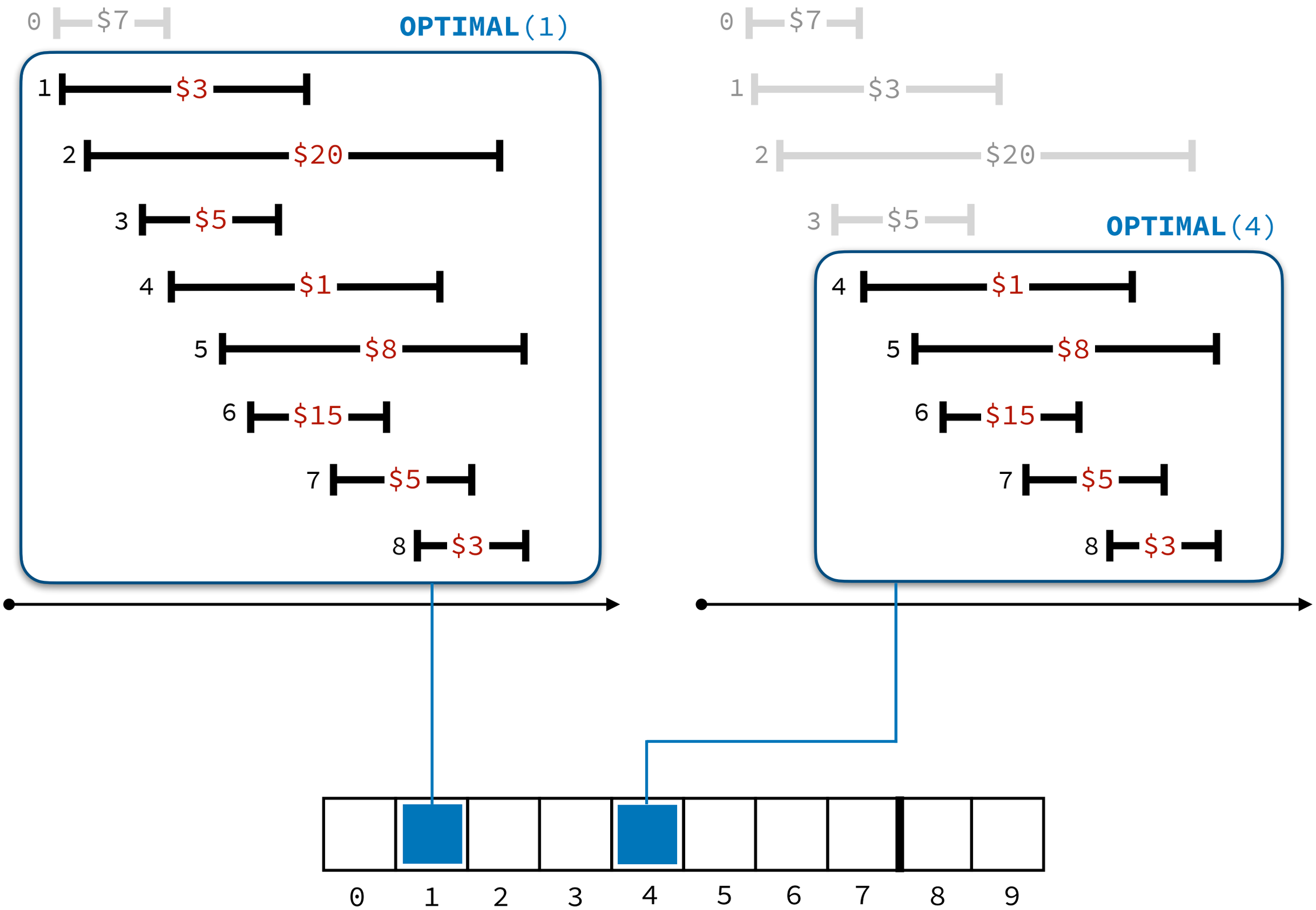
```
OPT(0) = MAX($7 + OPT(1),
             $0 + OPT(1))
```

Choice 1: Take activity `0` and solve **`OPT`**`(1)`

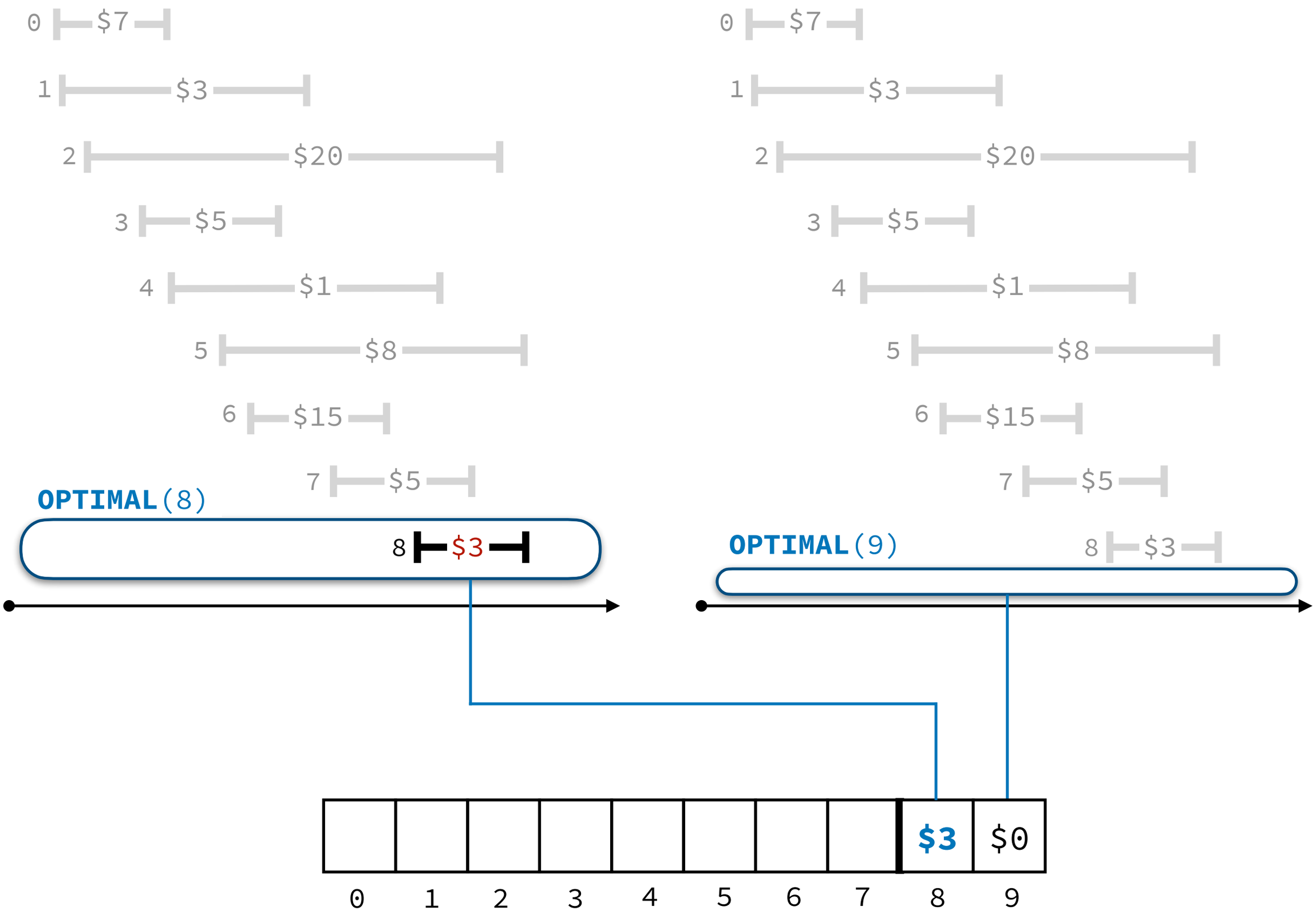
Choice 2: Don't take activity `0` and solve **`OPT`**`(1)`

There are overlapping subproblems!

4. Storing Solutions of Subproblems

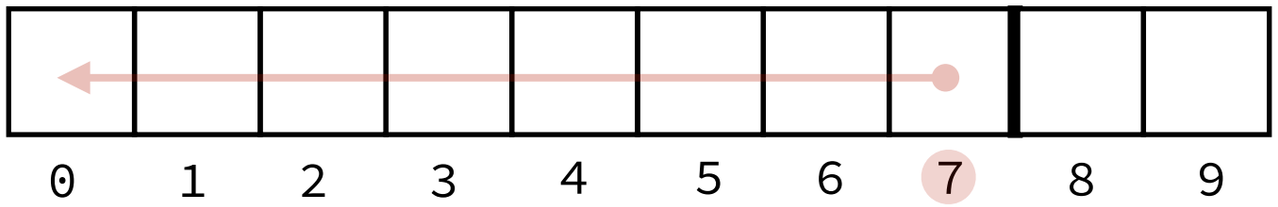


4. Storing Solutions of Subproblems



4. Implementing a Bottom-up Solution

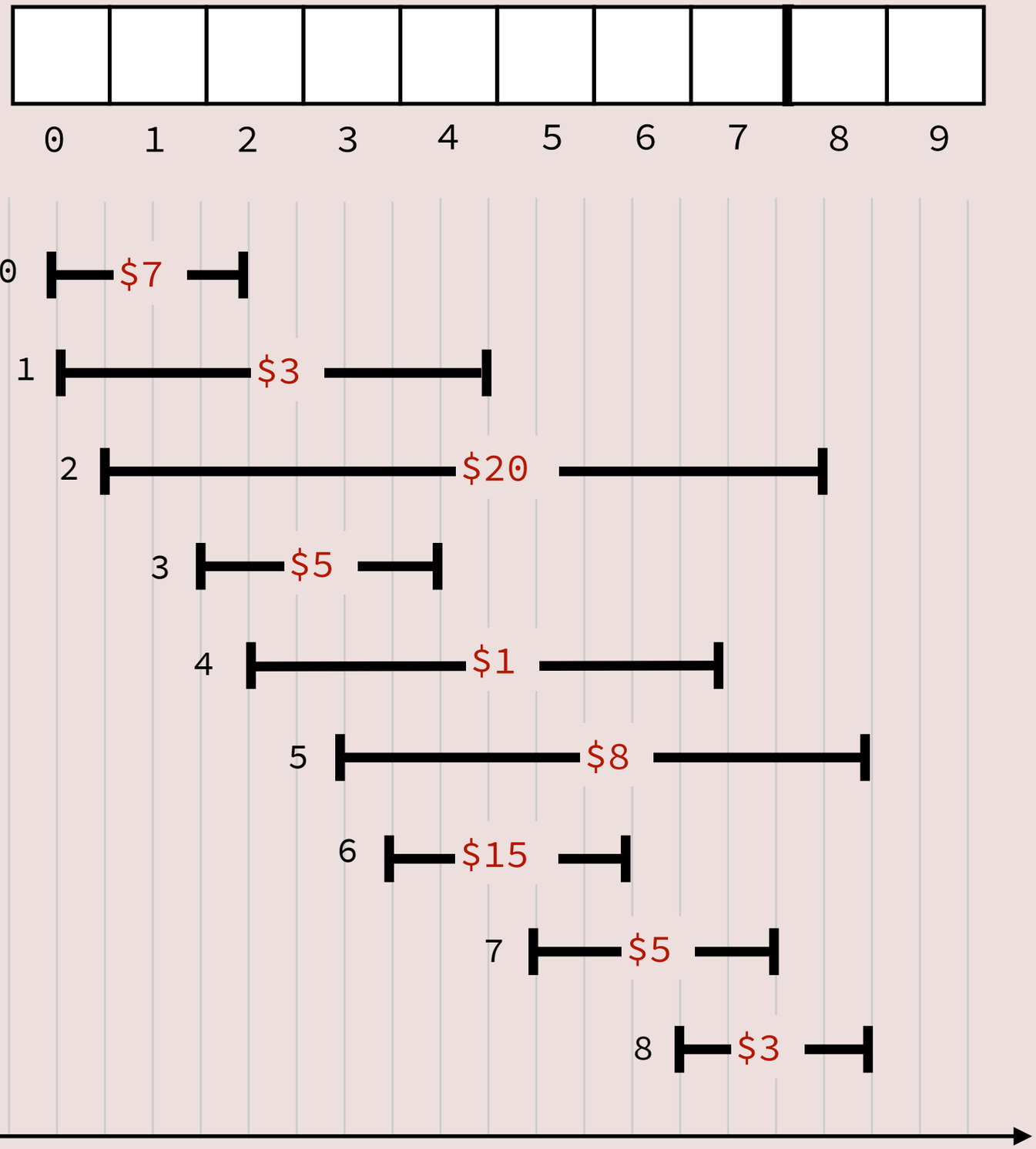
Solve from the smallest to the largest subproblem




```
ACTIVITY-SELECTION(A[], v[], N):

    SORT A[] based on start time

    CREATE result[N+1]
    result[N] = 0
    result[N-1]   = v[N-1]

    FOR i = N-2 TO 0:
        choice1 = v[i] + result[NEXT(i)]  // take
        choice2 = result[i+1]             // don't take

        result[i] = MAX(choice1, choice2)

    RETURN result[0]
```

Exercise

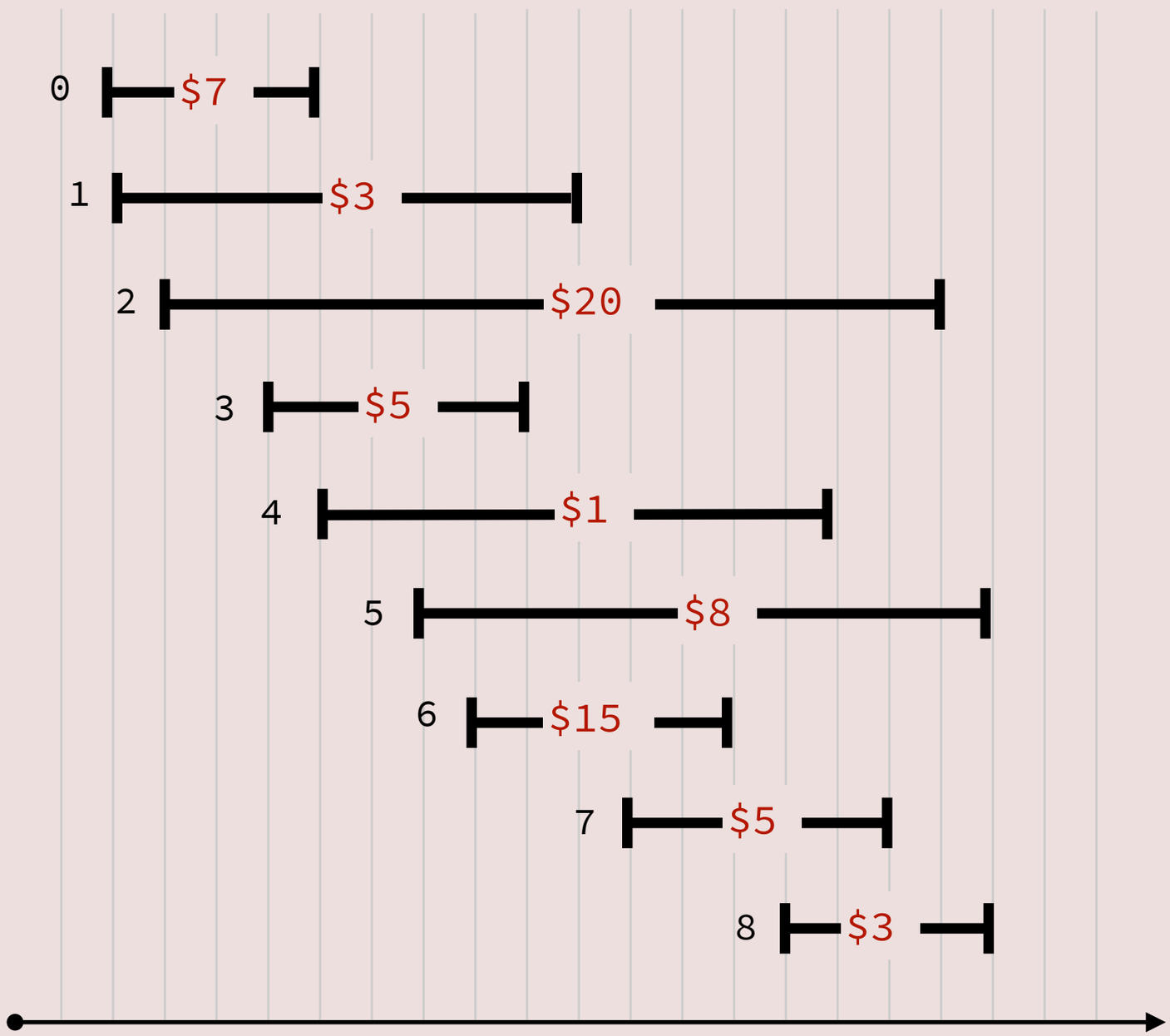
Show the contents of the array after running the bottom-up solution on the instance below.



Exercise

Show the contents of the array after running the bottom-up solution on the instance below.

25	20	20	18	18	18	18	5	3	0
0	1	2	3	4	5	6	7	8	9



5. Which Activities Did We Pick?

```
PRINT-ACTIVITIES(result[], N):
```

```
i = 0
WHILE (i <= N):
```

start here
and backtrack

25	20	20	18	18	18	18	5	3	0
0	1	2	3	4	5	6	7	8	9

5. Which Activities Did We Pick?

```
PRINT-ACTIVITIES(result[], N):
    i = 0
    WHILE (i <= N):
        IF (result[i] > result[i+1]):
```

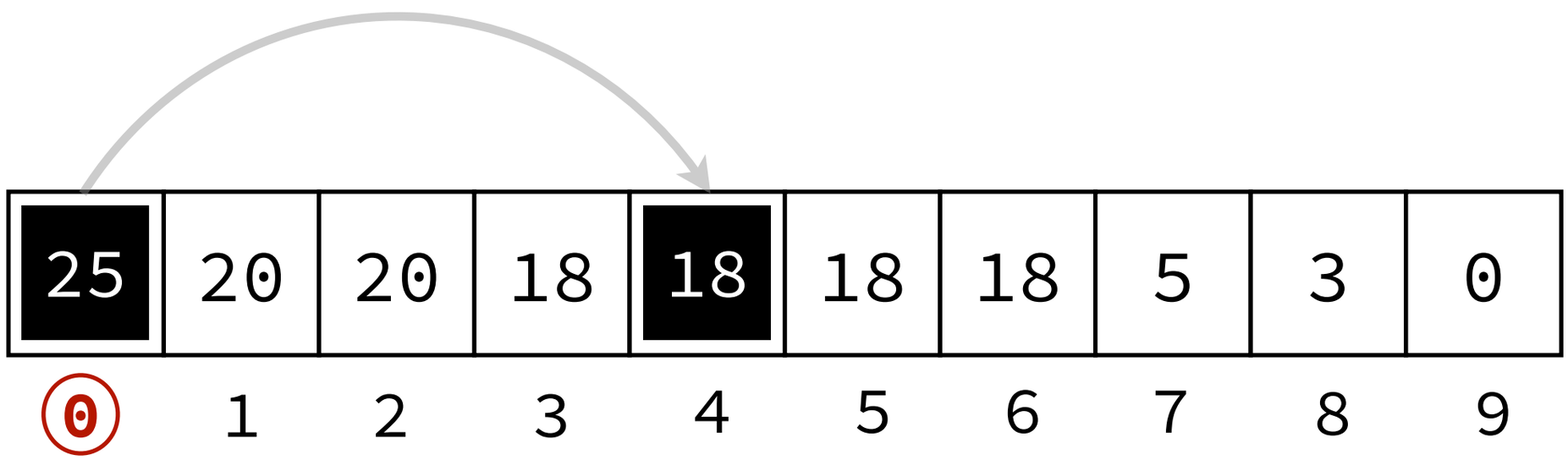
Take is better than **Don't take**

25	20	20	18	18	18	18	5	3	0
0	1	2	3	4	5	6	7	8	9

5. Which Activities Did We Pick?

```
PRINT-ACTIVITIES(result[], N):
    i = 0
    WHILE (i <= N):
        IF (result[i] > result[i+1]):
            PRINT i
            i = NEXT(i)
```

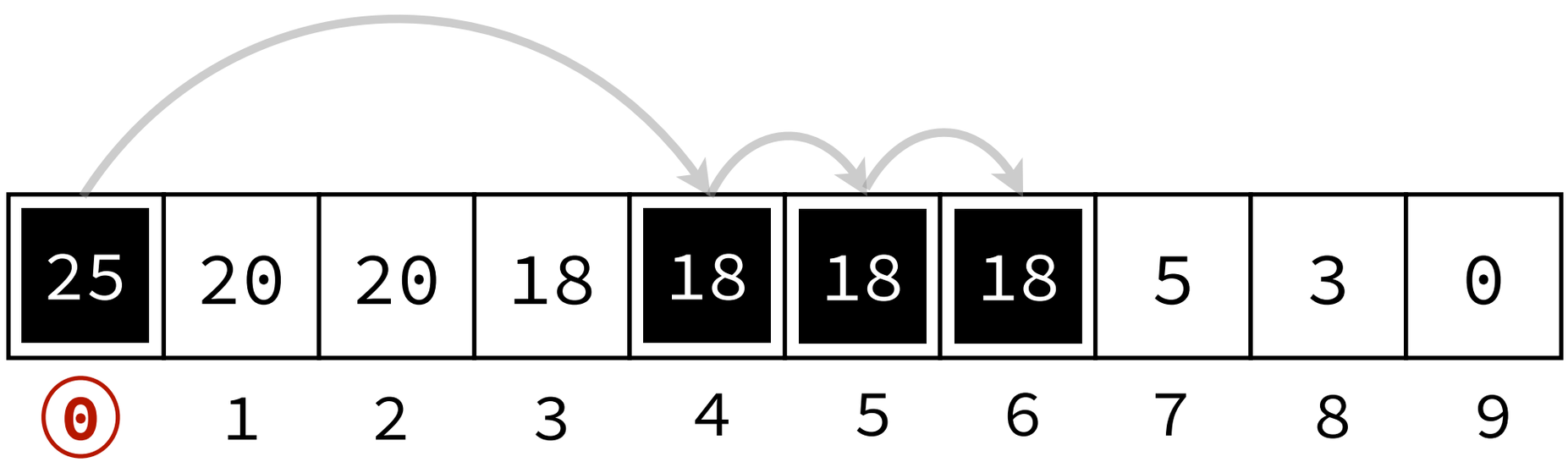
Report activity *i* and skip activities we can't take

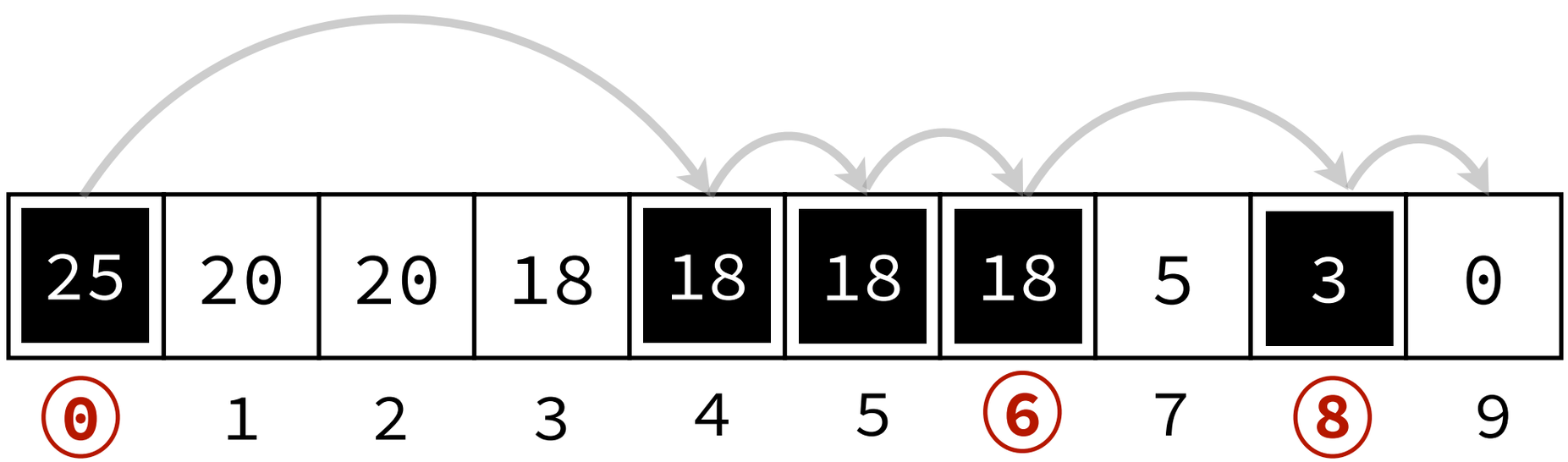


5. Which Activities Did We Pick?

```
PRINT-ACTIVITIES(result[], N):
    i = 0
    WHILE (i <= N):
        IF (result[i] > result[i+1]):
            PRINT i
            i = NEXT(i)
        ELSE
            i += 1
```

Don't take is better



5. Which Activities Did We Pick?

```
PRINT-ACTIVITIES(result[], N):
    i = 0
    WHILE (i <= N):
        IF (result[i] > result[i+1]):
            PRINT i
            i = NEXT(i)
        ELSE
            i += 1
```